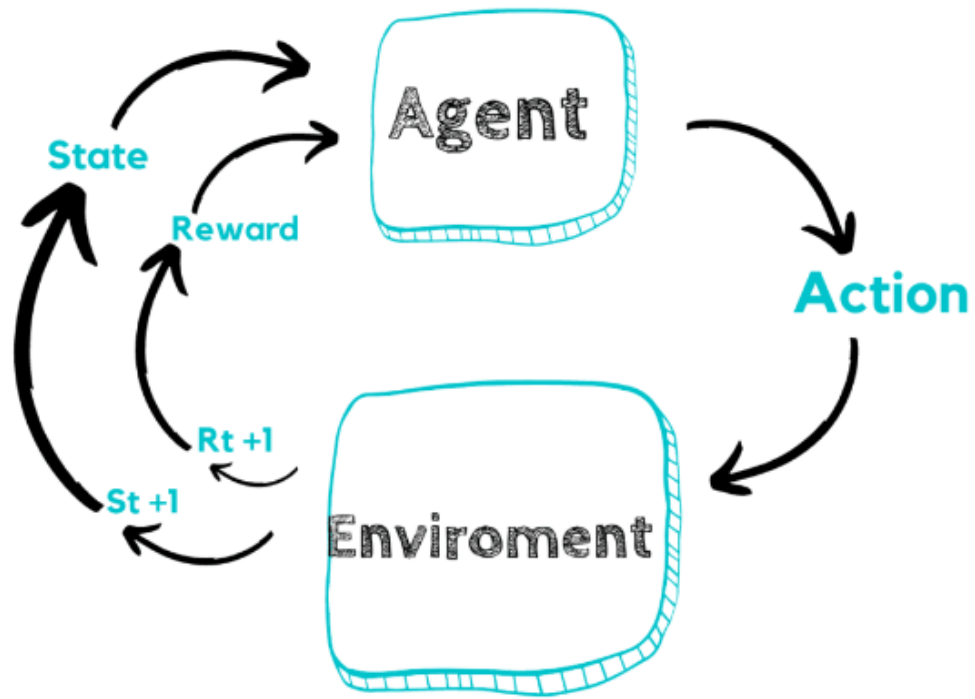


Reinforcement Learning

The Policy Gradient Theorem
&
REINFORCE

Reinforcement Learning



Different model-free approaches:

-**Value-based:** estimate

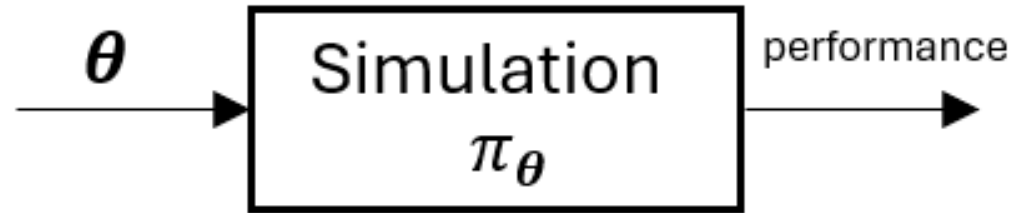
$$V(s) \quad \text{or} \quad Q(s,a)$$

-**Policy-based:** directly optimize the policy

$$\pi_{\theta}(a|s) \quad \theta \in R^d$$

Policy Gradient Search

- The Agent's policy in terms of probability $\pi_{\theta}(a|s)$:



- Assumptions:
 - (1) Episodic task
 - (2) NO γ (not mandatory)

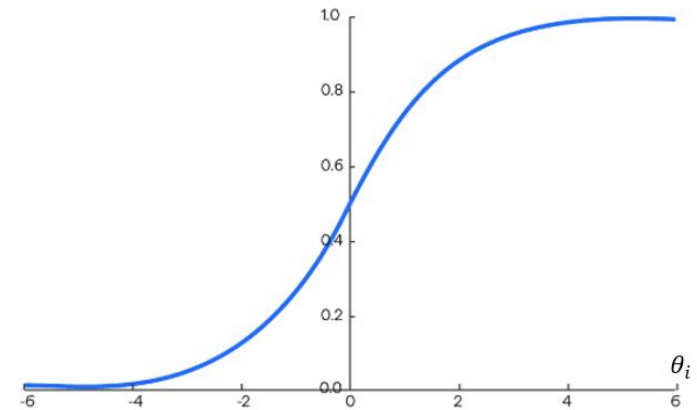
Policy Gradient Search

Soft-max parametrized policy:

$$\pi(a|s, \boldsymbol{\theta}) = \frac{e^{\psi(s,a,\boldsymbol{\theta})}}{\sum_b e^{\psi(s,b,\boldsymbol{\theta})}}$$

considering the linear model:

$$\psi(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^T \boldsymbol{\phi}(s, a)$$



$\boldsymbol{\theta}$ = vector of parameters
 $\boldsymbol{\phi}$ = feature vector

Note: similar to multinomial logistic regression

Policy Gradient Search

- Maximize the performance measure:

$$\max_{\theta} J(\theta) \quad \text{where} \quad J(\theta) = V_{\pi_{\theta}}(s)$$

- Gradient-based optimization:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

Note: We assume differentiability with respect to the coefficients.

Policy Gradient Theorem

THEOREM: For an episodic task:

$$\nabla J(\boldsymbol{\theta}) \propto \sum_s \mu(s) \left(\sum_a Q_{\pi_{\boldsymbol{\theta}}}(s, a) \nabla \pi(a|s, \boldsymbol{\theta}) \right)$$

$\mu(s)$ = state probability

$Q_{\pi_{\boldsymbol{\theta}}}$ = state – action value under $\pi_{\boldsymbol{\theta}}$

REINFORCE

- Taking only a single state S_t

$$\nabla J(\boldsymbol{\theta}) \propto \sum_a Q_{\pi}(S_t, a) \nabla \pi(a|S_t, \boldsymbol{\theta}) \xrightarrow{\frac{\pi(a|S_t, \boldsymbol{\theta})}{\pi(a|S_t, \boldsymbol{\theta})}} \sum_a \pi(a|S_t, \boldsymbol{\theta}) Q_{\pi}(S_t, a) \frac{\nabla \pi(a|S_t, \boldsymbol{\theta})}{\pi(a|S_t, \boldsymbol{\theta})}$$
$$\sum_a \pi(a|S_t, \boldsymbol{\theta}) Q_{\pi}(S_t, a) \nabla \log(\pi(a|S_t, \boldsymbol{\theta}))$$

- Taking only a single action A_t :

$$\nabla J(\boldsymbol{\theta}) \propto Q_{\pi}(S_t, A_t) \nabla \log(\pi(A_t|S_t, \boldsymbol{\theta}))$$

REINFORCE

- We use the observed reward $\hat{G}_t$ as an estimator:

$$Q_{\pi}(S_t, A_t) = \mathbb{E}_{\pi}[G_t | S_t, A_t] \approx \hat{G}_t(S_t, A_t)$$

→ Montecarlo step

- The (stochastic) gradient step becomes:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \hat{G}_t \nabla \log(\pi(A_t | S_t, \boldsymbol{\theta}_t))$$

→ REINFORCE learning step

Note: The logarithm of the softmax function is easy to calculate!

REINFORCE - Implementation

Input: $\pi_o(a|s, \theta)$, a , $\theta \in R^d$

Loop 'forever':

generate $(S_0, A_0, R_1, S_1, \dots, S_{T-1}, A_{T-1}, R_T)$ following $\pi(a|s, \theta)$

loop $\forall t = 0, 1, \dots, T - 1$ of an episode:

$$G \leftarrow \sum_{k=t+1}^T R_k$$

$$\theta \leftarrow \theta + \alpha G \nabla \log(\pi(A_t | S_t, \theta))$$

Example – Small Grid World

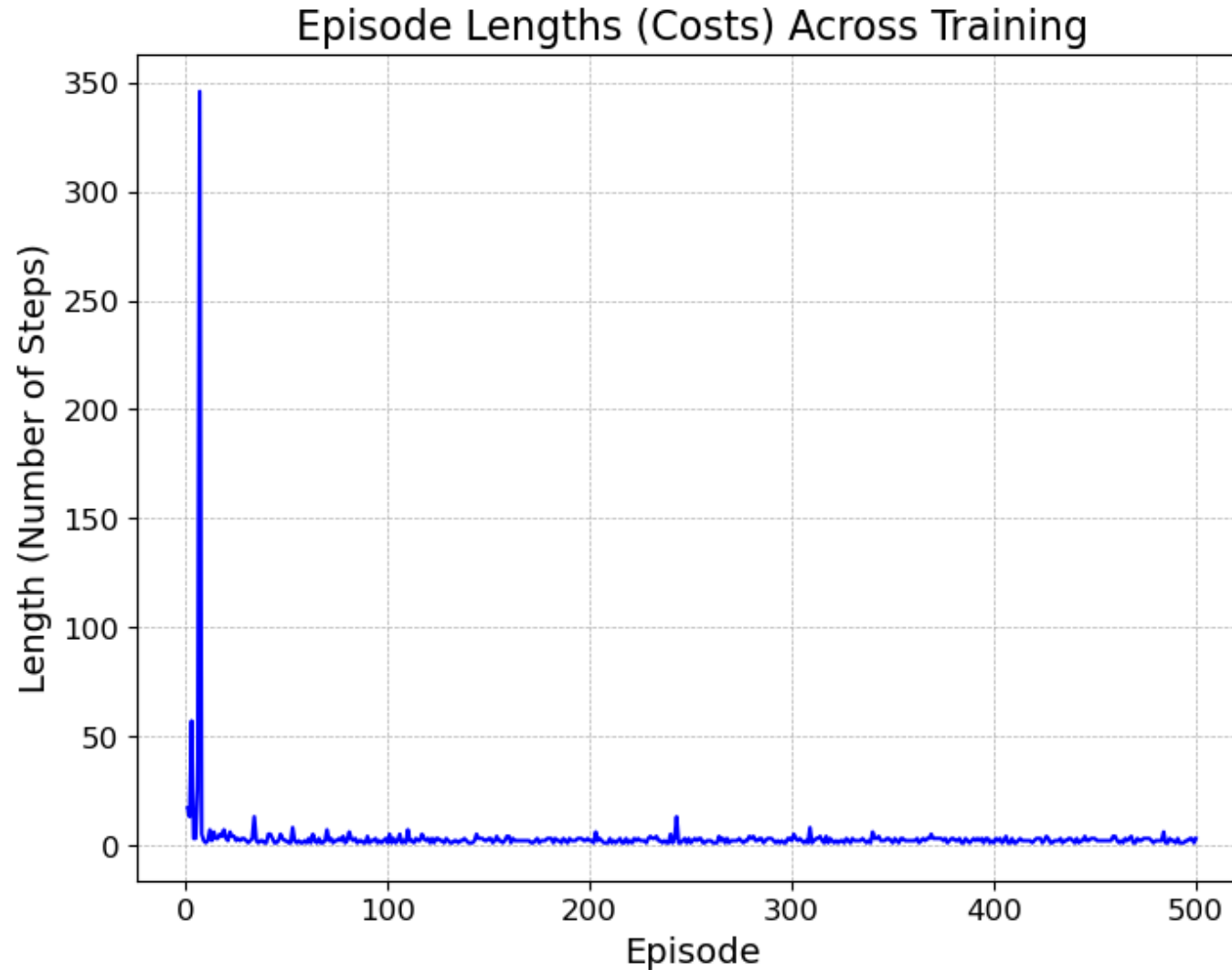
	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

- Movement cost = **-1**
- The agent wants to reach one of the **2 final goals** as fast as possible
- Features: $\phi = (x, y, d_{M1}, d_{M2})$

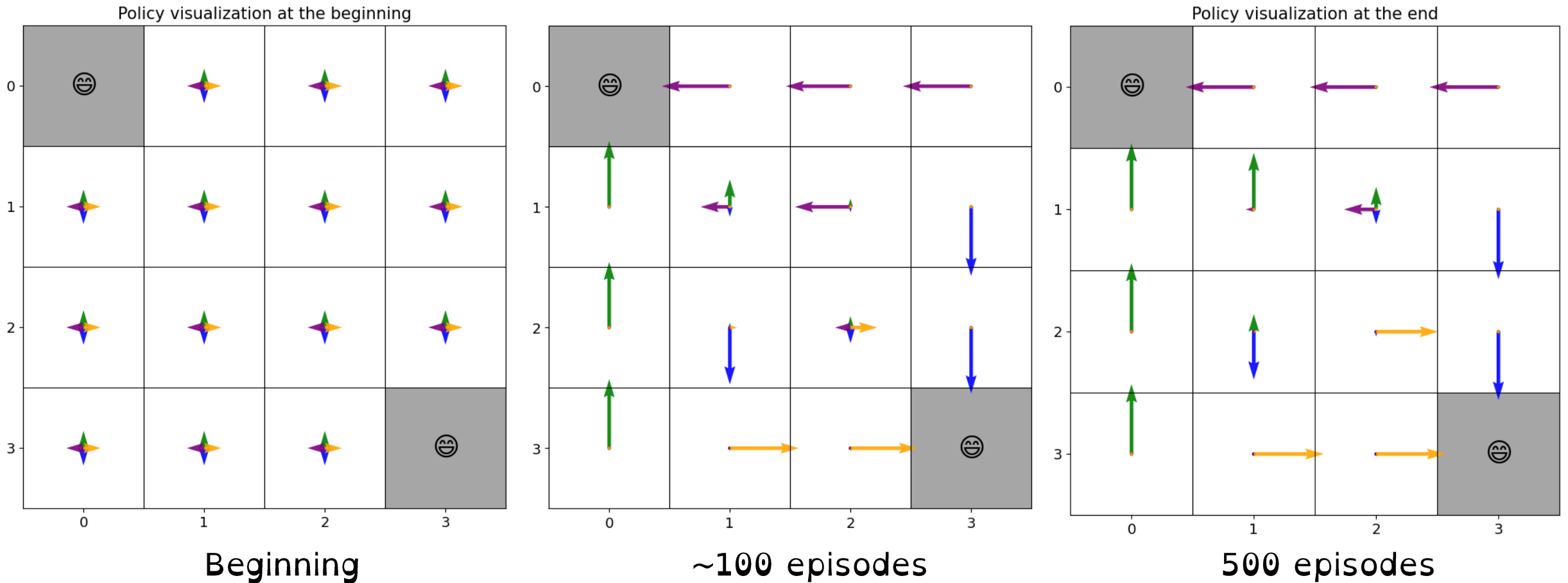
$(x, y) = \text{position}$

$d_{M1}, d_{M2} = \text{Manhattan distances to the goals}$

Example – Small Grid World

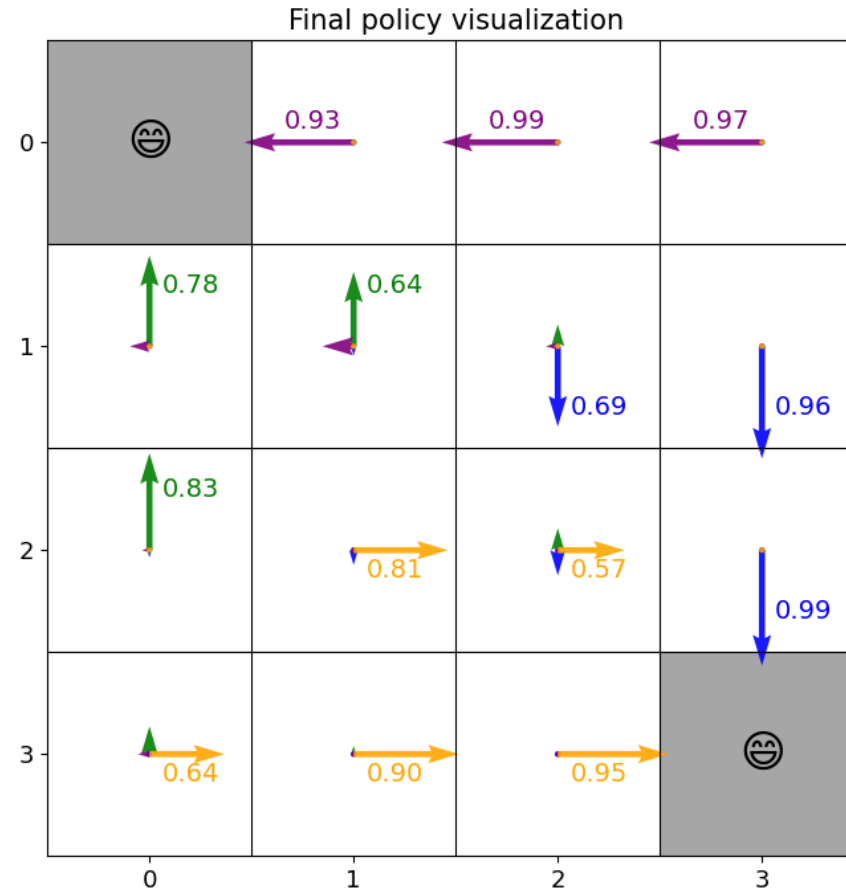
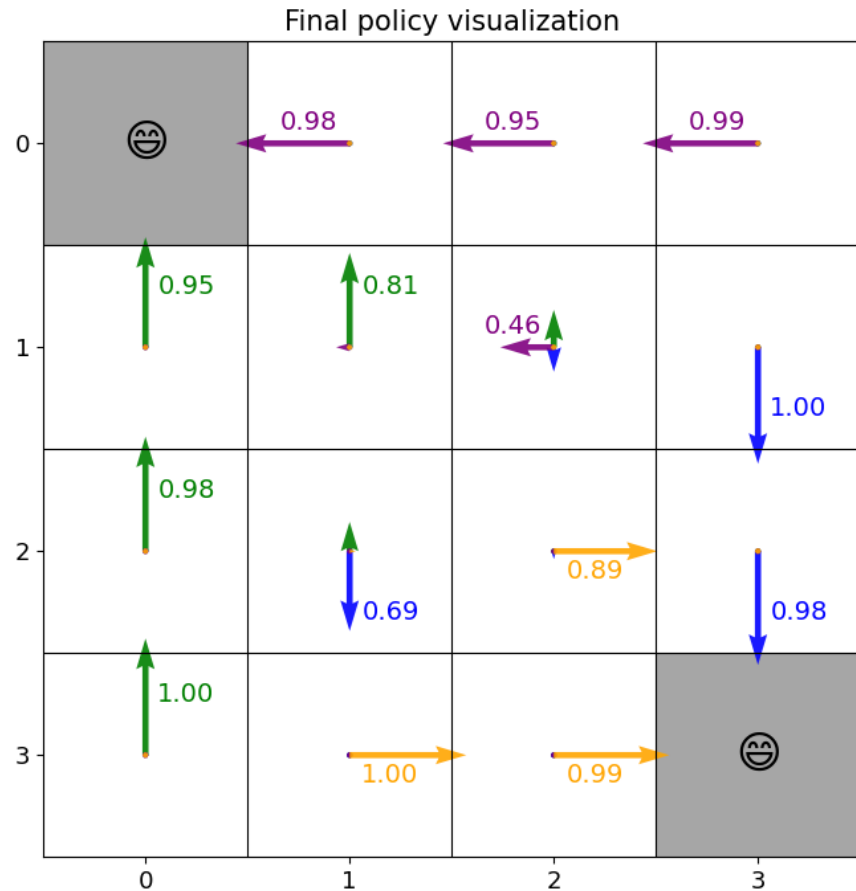


Example – Small Grid World



Note: As the agent learns, it may favor one direction over others, even if they are equally valid.

Example – Small Grid World

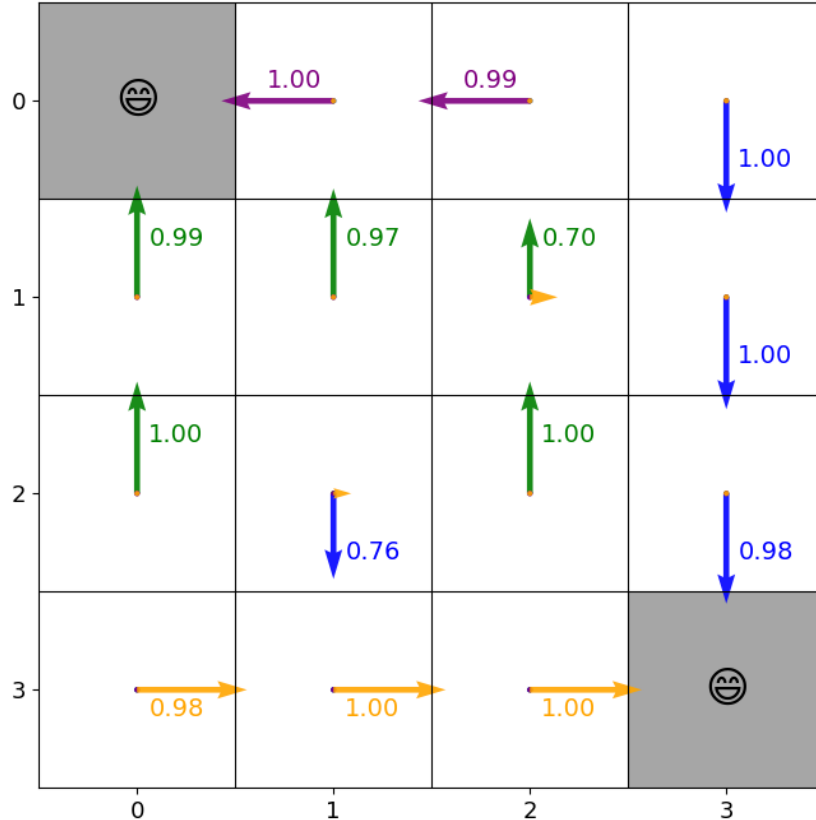


**HIGH
VARIANCE!**

Results after 500 episodes with 2 different seeds.

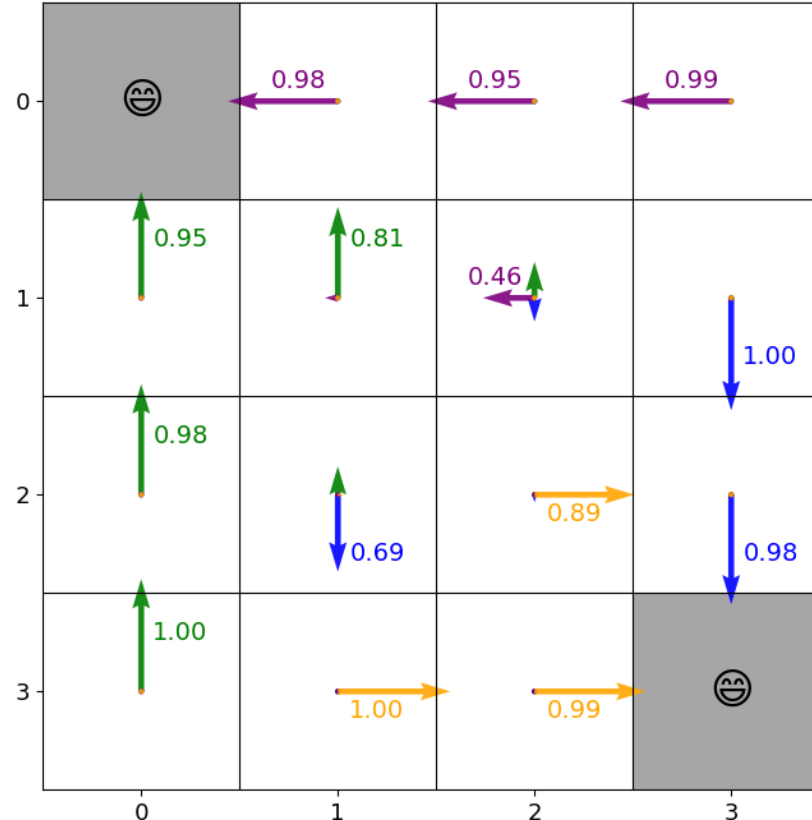
Example – Small Grid World

Final policy visualization



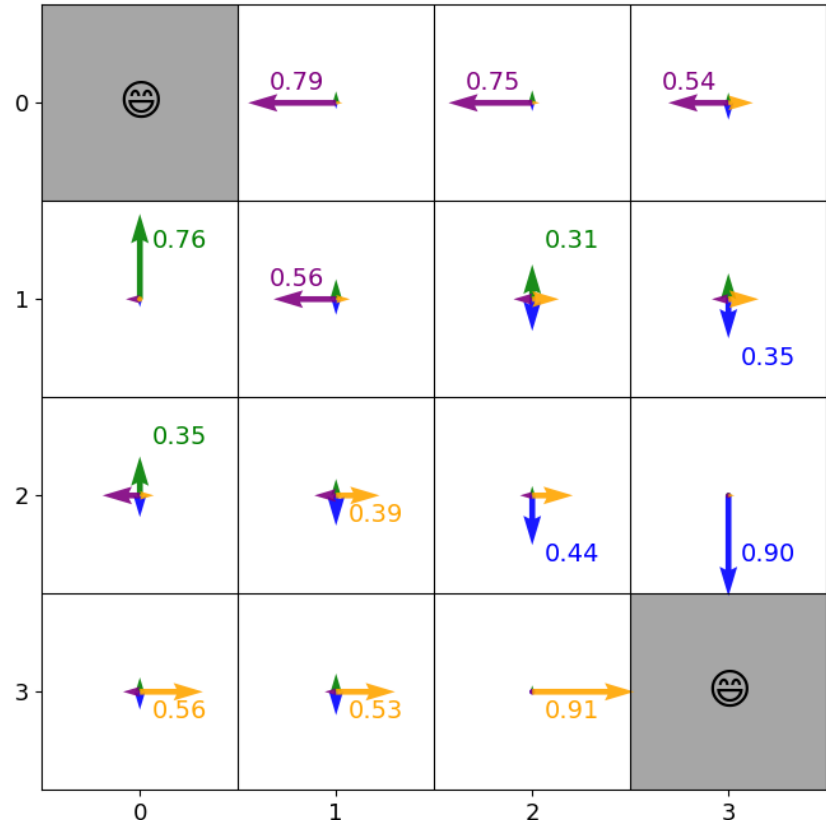
$\alpha = 0.04$

Final policy visualization



$\alpha = 0.01$

Final policy visualization



$\alpha = 0.001$

Too high α can cause overshooting, and even higher α leads to slow or non-convergence.

A too-low learning rate α results in slow learning and slower convergence.

Thank you for your attention!



**Politecnico
di Torino**

Gabriele Pirilli, Vida Gallo